

Examen 2017/18-2

Assignatura	Codi	Data	Hora inici
Anàlisi i disseny amb patrons	05.586	16/06/2018	15:30

05.586R16R06R18REEO€
05.586 16 06 18 EX

Enganxeu en aquest espai una etiqueta identificativa
amb el vostre codi personal
Examen

Aquest enunciat correspon també a les assignatures següents:

- 06.547 - Anàlisi i disseny amb patrons

Fitxa tècnica de l'examen

- Comprova que el codi i el nom de l'assignatura corresponen a l'assignatura matriculada.
- Només has d'enganxar una etiqueta d'estudiant a l'espai corresponent d'aquest full.
- No es poden adjuntar fulls addicionals, ni realitzar l'examen en llapis o retolador gruixut.
- Temps total: **2 hores** Valor de cada pregunta: **Indicat a la pregunta**
- En cas que els estudiants puguin consultar algun material durant l'examen, quins són?
CAP En cas de poder fer servir calculadora, de quin tipus? **CAP**
- Si hi hagi preguntes tipus test: Descompten les respostes errònies? **SÍ** Quant? **Indicat a la pregunta**
- Indicacions específiques per a la realització d'aquest examen:
CAP

Enunciats

Examen 2017/18-2

Assignatura	Codi	Data	Hora inici
Anàlisi i disseny amb patrons	05.586	16/06/2018	15:30

Pregunta 1 (10%)

Explica, en un màxim de cinc línies, què és un patró d'anàlisi i en què es diferencia d'un patró de disseny.

Solució:

Mòdul 1. Apartat 2.1

Pregunta 2 (20%)

Explica breument els problemes que intenten resoldre el patrons Dependency Injection i Estat.

Solució:

Dependency Injection: Mòdul 2, apartat 4.2

State: Mòdul 2, apartat 6.1

Pregunta 3 (10%)

Responen si són certes o falses les següents afirmacions. No cal que justifiqueu la vostra resposta. Cadascuna compta 2,5% si s'encerta i descompta 2,5% si es falla. Les respostes en blanc no compten ni descompten punts. La nota mínima d'aquesta pregunta serà 0.

- a) El patró associació històrica és un patró de disseny
(Fals. Mòdul 2, apartat 3.1)
- b) Si existeix un patró per un problema que tenim, no necessitem plantejar altres solucions ja que sabem que aquesta és la millor solució possible per al problema
(Fals. Mòdul 1, apartat 1.6)
- c) Una classe amb acoblament alt és més fàcil d'entendre de manera aïllada
(Fals. Mòdul2, apartat 2.1)
- d) La llei de Demèter ens ajuda a mantenir acoblament baix
(Cert. Mòdul 2, apartat 3.1)

Examen 2017/18-2

Assignatura	Codi	Data	Hora inici
Anàlisi i disseny amb patrons	05.586	16/06/2018	15:30

Exercici 1 (30%)

Suposem que volem desenvolupar un programari que registri els consums de combustibles d'una flota de vehicles industrials. Per a fer-ho, cada vehicle tindrà registrats els repostatges de combustible que fa (volum de combustible, estació de repostatge, data i hora).

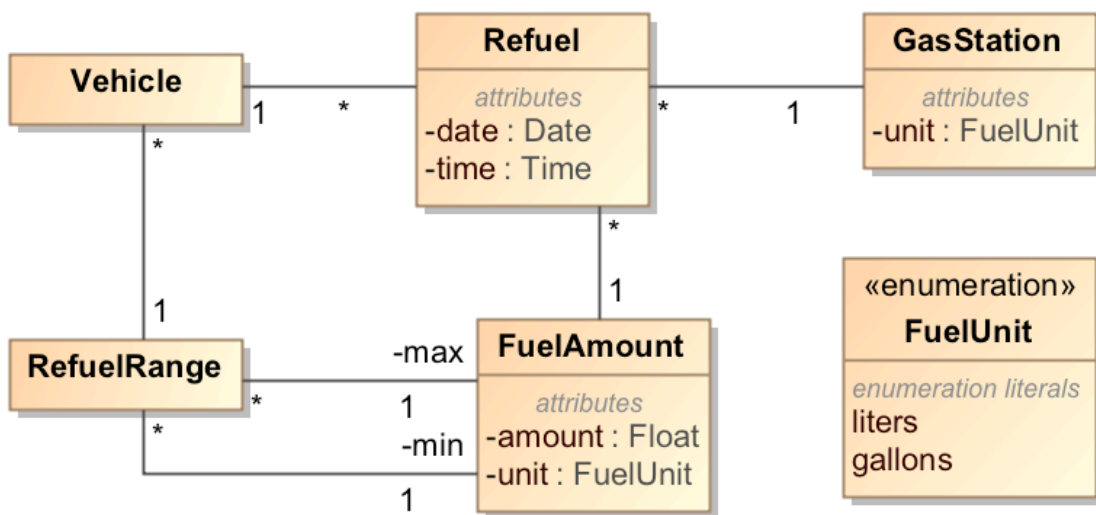
També volem controlar que un vehicle no pugui tenir associat un repostatge més gran que la capacitat del vehicle (per evitar el frau) i que mai es faci un repostatge de menys del 30% de la capacitat màxima del vehicle (per evitar tenir el vehicle aturat innecessàriament). Així, si un vehicle té un dipòsit de 100 litres de capacitat, mai hauria de tenir un repostatge de menys de 30 litres ni de més de 100 litres de combustible.

Per motius que ara no venen al cas però que ens complicaran una mica la vida, algunes estacions de repostatge reporten el volum en litres i altres en galons, així que haurem de fer les conversions necessàries.

Es demana el **diagrama estàtic d'anàlisi** per tal d'enregistrar aquesta informació. En cas que hagi aplicat algun patró d'anàlisi, indica quin o quins patrons has aplicat. Fes els supòsits que creguis necessaris i justifica les teves decisions.

Solució:

Hem aplicat el patró **Quantity** per a la quantitat de combustible (així evitem problemes amb les diferents unitats) i el patró **Range** al rang de quantitats de combustible que ha de tenir associat un vehicle:



Examen 2017/18-2

Assignatura	Codi	Data	Hora inici
Anàlisi i disseny amb patrons	05.586	16/06/2018	15:30

```
class Space {
    private Space next;
    private SpaceState space;

    public Boolean put(Piece piece) {
        space.put(piece);
        return true;
    }

    private void setState(state: SpaceState) {
        this.state = state;
    }
}

class FreeSpace extends SpaceState {
    override public Boolean canPass() {
        return true;
    }

    override public Boolean put(Piece piece) {
        OccupiedSpace nextState = new OccupiedSpace(piece);
        getSpace().setState(nextState);
        return true;
    }
}
```